

NEBIUS x NVIDIA GLOBAL AI HACKATHON

CAUVERIS

Where Failures Meet Their Cause.

Autonomous failure reconstruction, causal experimentation, verified patch generation, and digital-twin validation for AI-powered robotic systems.

Product and Build Blueprint | Version 3.0 | 20 September 2026

Core promise Cauveris rewinds a cloud-to-robot failure, launches counterfactual experiments in isolated Nebius branches, forges complete candidate patches with NVIDIA Nemotron, and returns only the fix that survives regression, performance, and robot-safety verification.

1. Executive Decision

Cauveris combines the strongest elements of autonomous production incident investigation and Physical AI debugging. It is neither a chatbot nor a simulator showcase. It is an engineering product for failures that cross cloud services, GPU inference, networks, ROS 2 middleware, robot control loops, and safety systems.

Golden demonstration incident

A warehouse robot consumes detections from a remote GPU inference service. Deployment v42 increases the dynamic batching window. Average latency remains acceptable, but tail latency rises. A delayed detection arrives outside the robot freshness budget, transform lookup fails, the controller misses a deadline, and the safety monitor triggers an emergency stop.

```
deployment v42 -> batching window rises -> inference P99 rises -> message age rises  
-> stale detection -> TF lookup failure -> control deadline miss -> emergency stop
```

Winning proof

- The failure is reconstructed from validated evidence.
- Competing explanations are expressed as falsifiable hypotheses.
- Nebius branches run controlled interventions.
- The causal branch reproduces the failure.
- Cauveris creates multiple applicable patch candidates and a regression test.
- Unsafe or ineffective fixes are rejected.
- The winning patch proves fail-before, pass-after behavior and preserves safety.

2. Product Users and Impact

User	Pain	Cauveris outcome
Robotics engineer	Intermittent ROS failures	Exact replay and timing isolation
SRE / platform engineer	Healthy averages hide tail failures	Deployment-to-robot causal chain
Inference engineer	Batching changes latency	Throughput versus freshness evidence
QA engineer	Field incidents are not tests	Permanent digital-twin regression test
Fleet operator	Stops lack explanation	Auditable report and next action

North-star metric

Primary metric Time from incident upload to a verified regression test that fails before the repair and passes after it.

- Time to first ranked hypothesis and controlled reproduction.
- Ground-truth root-cause rank and unsupported-claim rate.
- Failure rate before and after the patch.
- P50, P95, and P99 inference and control latency.
- Percentage of major claims linked to immutable artifacts.
- Human investigation steps avoided.

3. Inputs Given to Cauveris

Users may upload a structured incident bundle, connect supported sources, or load the signed golden demo incident. Raw evidence is validated and sanitized before model reasoning.

Input group	What is supplied	Demo example
Incident summary	System, observation, approximate time	Robot stopped near aisle B after deployment v42
Source and changes	Repositories, commit, deployment diff, build steps	Inference service and ROS navigation stack
Cloud evidence	Logs, OpenTelemetry traces, request metrics	Batching config and latency spans
Robot evidence	MCAP or rosbag, TF, odometry, commands, safety	Stale detections and emergency stop
Compute and network	GPU, CPU, queues, latency, jitter	GPU queue depth and network delay
Configuration	QoS, budgets, watchdogs, versions	120 ms freshness budget
Optional media	Incident or simulator video	Isaac Sim warehouse replay
Human constraints	Protected files, forbidden actions, budgets	Never weaken watchdog; max three patches

Incident bundle

```
incident-CAU-0001/  
  manifest.yaml  
  source/cloud-service/  
  source/robot-stack/  
  traces/otel.json  
  logs/service.jsonl  
  metrics/gpu.csv  
  metrics/network.csv  
  recordings/robot_run.mcap  
  config/inference.yaml  
  config/robot_params.yaml  
  deployments/events.json  
  observations/operator_note.md  
  media/incident.mp4
```

Input validation

- Checksums and immutable incident ID.
- Secret and credential scanning.
- Commit and software-version verification.
- Recording integrity and required-topic audit.
- Clock-offset estimation across cloud, host, robot, and simulation domains.
- Missing and contradictory evidence report.

4. What Cauveris Produces

Output	Artifact
Incident reconstruction	Synchronized timeline, clock map, evidence health score
Causal analysis	Ranked hypotheses, evidence, contradictors, falsifiers
Experiment record	Branches, interventions, seeds, logs, metrics, videos
Verified fix	Applicable patch and fail-before/pass-after test
Verification proof	Performance, safety, robustness, static analysis, rollback
Review package	Pull-request text, risk statement, checklist, report

Evidence vocabulary

State	Meaning
OBSERVED	Present in validated evidence
INFERRED	Model explanation, not yet reproduced
REPRODUCED	Controlled intervention recreated target failure
PATCHED	Candidate applied in an isolated branch
VERIFIED_IN_SANDBOX	All defined gates passed
PENDING_HARDWARE_VALIDATION	Physical deployment remains untested

5. How the Product Works

1. Validate the incident bundle and create an immutable incident.
2. Normalize cloud, GPU, network, ROS, and simulation clocks.
3. Build a unified event graph and identify the smallest suspicious window.
4. Map runtime evidence to source files, parameters, and recent changes.
5. Generate three to five falsifiable Nemotron hypotheses with evidence and contradictors.
6. Translate each hypothesis into a controlled intervention.
7. Checkpoint the clean build and fork one Nebius Sandbox branch per experiment.
8. Replay the incident in ROS 2 and Isaac Sim, collecting deterministic artifacts.
9. Rank branches by reproduction strength and falsification survival.
10. Freeze the causal branch and enter Patch Forge.
11. Generate multiple candidate fixes and a regression test.
12. Build and verify candidates independently.
13. Run expanded fixed-seed and randomized trials on the winner.
14. Produce the patch bundle and report for human approval.

Interaction principle Every stage creates something visible: a timeline, graph, ghost run, patch card, code diff, replay, or evidence artifact. The user never waits on an unexplained chat spinner.

6. Product Appearance and Visual Direction

Cauveris should look like a restrained engineering command center. The visual system uses a near-black indigo canvas, electric-cyan evidence paths, amber warnings, soft green verification states, and off-white text. Motion is used to explain causality, not decorate the interface.

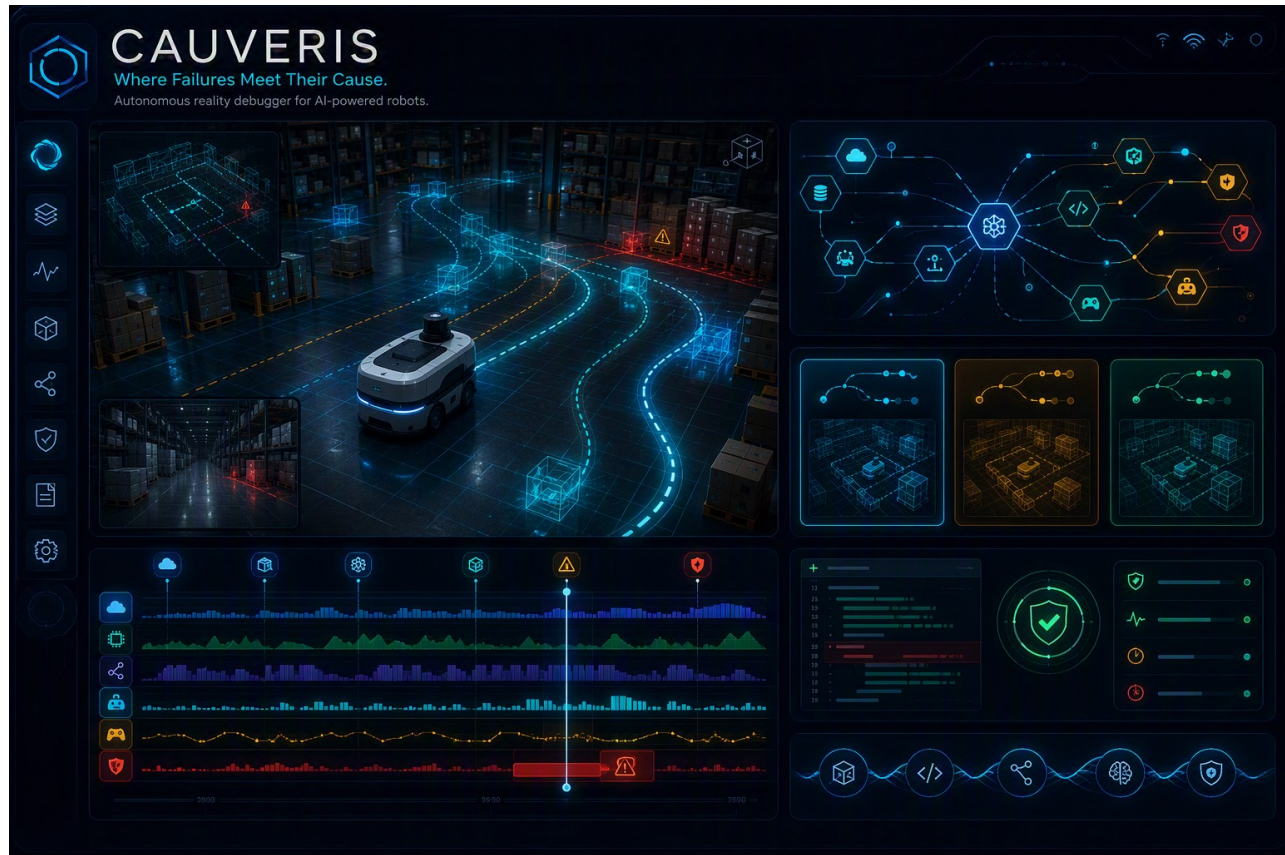


Figure 1. Cauveris Mission Control concept: digital twin, causal graph, experiment branches, and synchronized cloud-to-robot timeline. Visual generated with Microsoft Copilot.

Mission Control composition

Region	Visible content	Purpose
Top command bar	Incident stage, model route, budget, Judge Mode	Instant orientation
Center stage	Isaac Sim or recorded 3D digital twin	Physical consequence remains central
Bottom timeline	Cloud, GPU, network, ROS, controller, safety lanes	Cross-domain synchronization
Left navigation	Rewind, Constellation, Ghost Lab, Patch Forge, Vault	Clear investigation journey
Right evidence rail	Event, source, confidence, contradiction, artifact	Auditable conclusions
Status ribbon	Observed through verified states	Honest certainty

7. Screen-by-Screen Product Specification

Screen	Hero visual	Primary action	Completion signal
Mission Control	Robot twin with incident pulse	Load incident and reconstruct	Evidence ready
Reality Rewind	Scene plus synchronized time scrubber	Rewind stop to deployment	Failure window locked
Causal Constellation	Animated cause-effect graph	Open evidence and falsifiers	Testable hypotheses
Ghost Lab	Parallel translucent robot paths	Run and compare interventions	Failure reproduced
Patch Forge	Candidate bracket and live diff	Inspect, reject, verify	One patch survives
Victory Replay	Before/after split screen	Replay identical conditions	Zero target failures
Evidence Vault	Claims connected to artifacts	Export review package	All claims evidenced

Signature visual moments

- Reality Rewind moves backward from the emergency stop to the responsible deployment.
- Ghost Lab overlays several counterfactual robot trajectories.
- Causality Lock turns one branch green after repeatable reproduction.
- Patch Forge visibly eliminates unsafe and ineffective candidates.
- Victory Replay shows the pre-patch robot stopping while the patched robot completes the same route.

Feasible implementation

- Next.js, TypeScript, Tailwind, React Flow, ECharts, and Monaco.
- SSE for live experiment updates.
- Recorded WebM or lightweight Three.js replay if live Isaac streaming is unstable.
- Desktop-first 1440p Judge Mode and detailed Engineer Mode.
- Reduced-motion mode and color-independent state icons.

8. Patch Forge: Creating the Fixed Patch

A fixed patch is a complete change against the exact incident commit that applies cleanly, builds, adds a regression test, removes the reproduced failure, preserves safety, and includes rollback instructions.

15. Freeze the reproduced causal branch and smallest implicated code window.
16. Ask Nemotron Super for two or three constrained fixes with explicit assumptions.
17. Apply each candidate in an independent Sandbox using `git apply --check`.
18. Generate a regression test from the reproduction oracle.
19. Prove the test fails on the original commit.
20. Run unit, integration, replay, load, static-analysis, and safety suites.
21. Reject symptom hiding, blanket exception handling, test deletion, or watchdog weakening.
22. Score candidates on correctness, safety, regression strength, performance, minimality, and maintainability.
23. Run expanded verification on the highest-scoring survivor.
24. Create the signed review package.

```
PATCH_VERIFIED = applies_cleanly
AND regression_test_fails_before
AND regression_test_passes_after
AND original_failure_not_reproduced
AND existing_tests_pass
AND safety_invariants_pass
AND performance_within_budget
AND forbidden_change_scan_passes
AND rollback_test_passes
```

Patch package

```
patch-package/
fix.patch
regression_test.patch
verification.json
before_after/{traces,metrics,replay-videos,mcap-excerpts}/
PULL_REQUEST.md
reviewer-checklist.md
rollback.sh
```

Safety boundary Cauveris may create a working patch and prepare a pull request, but it never merges or deploys to a real robot without human approval.

9. Technical Architecture

CAUVERIS WEB APPLICATION

- > FastAPI Gateway + SSE
- > Incident, Artifact, Report services
- > Evidence Normalizer: OTel + logs + metrics + MCAP + Git
- > Agent Orchestrator: policies, budgets, approvals
 - > Nebius Token Factory: Nemotron router
 - > Token Factory Sandboxes: checkpoint, fork, execute, collect
 - > ROS 2 + NVIDIA Isaac Sim runner
- > PostgreSQL + object storage + evidence graph
- > Verification Engine: replay, tests, load, safety, rollback

Architectural rule

Models reason; tools establish facts. Nemotron never replaces MCAP parsing, Git diff, test execution, timestamp alignment, metric computation, or safety oracles. Model claims must cite artifact IDs.

State machine

RECEIVED -> VALIDATING -> NORMALIZING -> MAPPING -> HYPOTHESIZING
-> PLANNING -> RUNNING_BRANCHES -> SCORING -> PATCH_FORGE
-> VERIFYING -> AWAITING_REVIEW -> COMPLETED

10. Model Portfolio and Routing

Model or method	Role	Status
NVIDIA Nemotron 3 Nano	Classification, extraction, schema repair, UI explanations	Fast required route
NVIDIA Nemotron 3 Super	Hypotheses, experiments, patches, evidence arbitration	Primary reasoning route
NVIDIA Nemotron 3 Ultra	Hard cross-repository escalation and synthesis	Optional, budgeted
NVIDIA multimodal or Cosmos model	Video and physical-scene grounding	Stretch only
Open embedding model	Artifact retrieval and similar incidents	Optional
Classical anomaly models	Change points and latency outliers	Required non-LLM

```
extract/classify/summarize -> Nano
hypothesize/plan/generate-patch/review -> Super
repeated hard failure -> Ultra with strict budget
deterministic tool can answer -> no LLM call
no experimental support -> remain INFERRED
```

11. Nebius and NVIDIA Integration

Technology	Essential responsibility
Nebius Token Factory	Hosts Nemotron calls, structured reasoning, routing, usage telemetry
Token Factory Sandboxes	Clean checkpoints, branches, execution, rollback, artifacts
NVIDIA Isaac Sim	Judge-visible digital twin and repeatable physical test envelope
ROS 2 and MCAP	Realistic middleware evidence and deterministic replay
OpenTelemetry	Connects cloud inference spans with robot-side events

The core replay and safety oracle should work headlessly in ROS 2 before Isaac Sim is added. Gazebo or recorded visualization remains the schedule-safe fallback if live simulation risks the core experiment loop.

12. Open-Source Stack and Repository

Layer	Recommended choice
Frontend	Next.js, TypeScript, Tailwind, React Flow, ECharts, Monaco
API	FastAPI, Pydantic, SSE
Data	PostgreSQL, pgvector, MinIO
Evidence	OpenTelemetry, MCAP, rosbag2, Polars
Source mapping	ripgrep, tree-sitter, GitPython
Testing	pytest, colcon test, k6/Locust, ruff, mypy, Bandit
Simulation	ROS 2 Jazzy, Isaac Sim; Gazebo fallback
Packaging	Docker Compose and pinned OCI images
License	Apache-2.0

```
cauveris/  
  apps/{web,api}/  
  services/{ingestion,timeline,repository_mapper,hypothesis_agent}/  
  services/{experiment_planner,sandbox_controller,patch_agent,verifier}/  
  packages/{schemas,model_gateway,artifact_store}/  
  robotics/{demo_robot,fault_injectors,safety_oracles,simulation}/  
  scenarios/cloud_to_robot_latency/  
  evaluations/golden_incidents/  
  tests/{unit,integration,end_to_end,safety}/  
  docs/
```

13. Security and Safety

Threat	Control
Prompt injection in evidence	Treat repository and logs as untrusted data
Malicious build scripts	Sandbox, network off, resource and time limits
Secret leakage	Scan and redact before model context
Destructive command	Allowlist, policy, writable experiment area only
Safety weakening patch	Protected files, invariant tests, independent review
False causal claim	Intervention required before reproduced or verified
Demo misrepresentation	Label cached and stored artifacts clearly

- A genuine obstacle must still trigger a stop.
- Sensor disconnection must produce a safe state.
- Stale detections must never be treated as fresh.
- Protected watchdog behavior cannot be weakened.
- Rollback must restore the original branch and artifacts.
- Physical deployment remains pending supervised validation.

14. Evaluation Gates

Metric	Target
Ground-truth cause	Top two
Pre-patch reproduction	At least 70%
Post-patch target failure	0 of 100 stored trials
Regression test	Fails before and passes after
Critical safety suite	100% pass
P95 control latency	No worse than 10%; improvement preferred
Major claims with artifacts	100%
Unsupported claims	Below 5%
Short live path	Under three minutes

Baselines

- Raw log inspection.
- Single model prompt without tools.
- Static tests only.
- Cauveris without branch experiments.
- Full Cauveris.

15. Three-Minute Demonstration

Time	Visual	Message
0:00-0:18	Robot stops in warehouse	A deployment caused a rare cross-domain failure
0:18-0:35	Load CAU-0001	Source, traces, GPU metrics, ROS, config, observation
0:35-0:58	Reality Rewind	Deployment to emergency stop is synchronized
0:58-1:22	Causal Constellation	Nemotron proposes falsifiable causes
1:22-1:45	Ghost Lab	Nebius branches test batching, QoS, clock skew
1:45-2:12	Patch Forge	Unsafe and slow candidates are rejected
2:12-2:42	Victory Replay	Before stops; after completes; 0/100 stored failures
2:42-3:00	Evidence Vault	Patch, test, report, and architecture

Exact demonstration sequence

25. Open Judge Mode and load CAU-0001.
26. Show input chips and evidence health.
27. Press Reconstruct.
28. Scrub backward from the stop to deployment v42.
29. Open the batching hypothesis with supporting and contradicting evidence.
30. Run three shortened Ghost Lab branches.
31. Show the branch reproducing 14 failures in 20 trials.
32. Open Patch Forge: candidate A fails safety, B fails latency, C passes.
33. Inspect the generated test and key diff.
34. Run ten live verification trials while showing stored 100-trial evidence.
35. Play split-screen Victory Replay.
36. Export the pull-request package.

Before patch: 14/20 failures
 After patch live: 0/10 failures
 After patch full evidence: 0/100 failures
 Regression tests: PASS
 Critical safety tests: PASS
 P95 control latency: IMPROVED
 Rollback: PASS

16. Thirty-Five-Day Build Roadmap

Phase	Days	Deliverable	Exit gate
Alignment	1-2	Repo, license, model and Sandbox smoke tests	UI shows Sandbox task
Ground truth	3-6	Cloud service, ROS pipeline, fault, recordings	Failure without AI
Ingestion	7-10	Parsers, clocks, evidence health, timeline	Deployment-to-stop visible
Reasoning	11-14	Mapper, Nemotron, hypotheses	Cause in top two
Branching	15-19	Checkpoints, experiments, Ghost Lab	Three branches run
Patch Forge	20-24	Candidates, regression test, policies	Fail-before/pass-after
Verification	25-28	Sweeps, safety, load, rollback, report	Targets reached
Product polish	29-31	Seven screens and Judge Mode	New user succeeds
Reliability	32-33	Golden set, injection tests, fallbacks	Five rehearsals
Submission	34-35	README, hosted demo, video, release	Checklist complete

Cut order

- 37. Remove multimodal scene reasoning.
- 38. Replace live Isaac streaming with recorded replay.
- 39. Use a lightweight state machine instead of a workflow platform.
- 40. Limit Patch Forge to two candidates.
- 41. Never cut deterministic reproduction, evidence links, fail-before/pass-after proof, or safety verification.

17. Team Plan and Healthy Collaboration

Ownership	Primary work	Backup responsibility
Agent and backend	Schemas, FastAPI, Nemotron, patch generation	Can run robot demo
Robotics and product	ROS 2, Isaac Sim, faults, dashboard	Can run backend demo
Shared	Architecture, safety policy, verification, submission	Both review and present

- Use Apache-2.0 and credit both contributors.
- Develop in short-lived feature branches and merge through reviewed pull requests.
- Protect main and require one approval where GitHub settings allow.
- Keep secrets out of Git and use separate credentials.
- Write down prize sharing, attribution, repository control, and post-hackathon intentions.
- Ensure both teammates can start and present the complete golden demo.

18. Definition of Done

- Prepared incident validates and builds the expected synchronized timeline.
- Nemotron produces schema-valid hypotheses with artifact references.
- At least three isolated experiment branches execute.
- The planted cause is reproduced and ranked in the top two.
- At least two complete patch candidates are independently tested.
- The generated regression test fails before and passes after.
- Expanded verification meets performance and safety thresholds.
- The dashboard separates observation, inference, reproduction, and verification.
- Every major claim links to evidence.
- Nebius and NVIDIA usage is visible and essential.
- The public repository includes Apache-2.0, setup, architecture, evaluation, security, and demo instructions.

Final strategic focus The simulator explains the consequence, the interface explains the evidence, the branches prove the cause, and Patch Forge proves the repair.

19. Immediate Actions

42. Create the Cauveris repository and add Apache-2.0.
43. Invite the teammate and document ownership and prize expectations.
44. Verify Nemotron model IDs available to the team account.
45. Complete Sandbox create, execute, fork, rollback, and artifact smoke tests.
46. Implement the deterministic cloud-to-robot failure before agent code.
47. Record healthy and faulty MCAP evidence.
48. Define the incident manifest and artifact URI format.
49. Build Reality Rewind before the complete dashboard.
50. Implement strict hypothesis JSON with evidence and falsifiers.
51. Establish fail-before/pass-after proof by Day 24.

References and Source Notes

[R1] Nebius x NVIDIA Global AI Hackathon
<https://nebiusglobalaihackathon.devpost.com/>

[R2] Official event rules
<https://nebiusglobalaihackathon.devpost.com/rules>

[R3] NVIDIA Nemotron on Nebius Token Factory
<https://nebius.com/services/token-factory/nemotron>

[R4] Nebius Token Factory
<https://nebius.com/services/token-factory>

[R5] NVIDIA Isaac Sim ROS 2 documentation
https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/index.html

[R6] NVIDIA Isaac Sim open-source repositories
<https://github.com/isaac-sim>

[R7] NVIDIA Isaac ROS
<https://developer.nvidia.com/isaac/ros>

Source basis: this blueprint synthesizes the uploaded AI Incident Commander and FactoryGhost roadmaps. Model availability and exact IDs can change and must be discovered through the team account. The interface image is a Microsoft Copilot-generated concept visual, not a completed-build screenshot.